



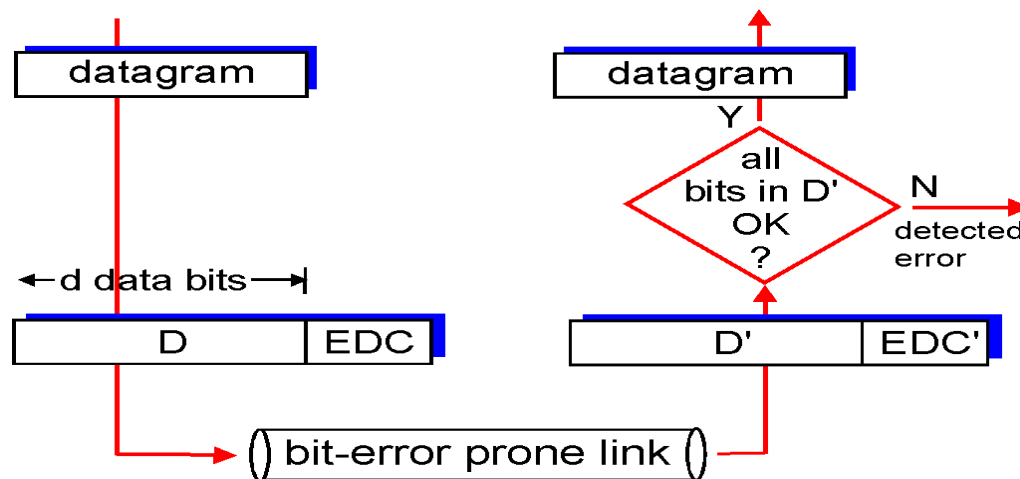
Wie können eigentlich Fehler erkannt und
ggf. sogar behoben werden?

Fehlererkennung

EDC = Error Detection and Correction bits (Redundante Information)

D = Daten, die durch EDC geschützt werden

- Durch Fehlererkennungs- und -korrekturmechanismen kann der Empfänger manchmal – aber nicht immer – erkennen, dass Bitfehler aufgetreten sind
- Höhere Redundanz hilft



Fehlererkennung auf Bitebene

- Idee für die Verwendung von Redundanz
 - Daten werden in Form eines Codes übertragen
 - Ausnutzung der „**Distanz**“ zwischen gültigen Codewörtern, d.h. **nicht alle möglichen Bitkombinationen sind gültige Codewörter**
- **Hamming-Abstand**
 - Anzahl unterschiedlicher Bits zweier Codewörter c_1 und c_2 , d.h. Anzahl der 1-Bits von $c_1 \text{ XOR } c_2$.
 - Beispiel: $d(10001001, 10110001) = 3$
 - Hamming-Abstand D von vollständigem Code C :

$$D(C) := \min \{ d(c_1, c_2) \mid c_1, c_2 \in C; c_1 \neq c_2 \}$$

Hamming – Codes (I)

- Hamming-Abstand eines Codes bestimmt die Fähigkeit des Codes, Fehler zu erkennen und zu beheben
 - **erkennen** von e -Bit-Fehlern: Hamming-Abstand $e+1$ notwendig
 - **beheben** von e -Bit-Fehlern: Hamming-Abstand $2e+1$ notwendig
- Wenn wir also einen Hamming-Abstand von 3 haben, können wir 2-Bit-Fehler erkennen und 1-Bit-Fehler beheben

Hamming – Codes (II)

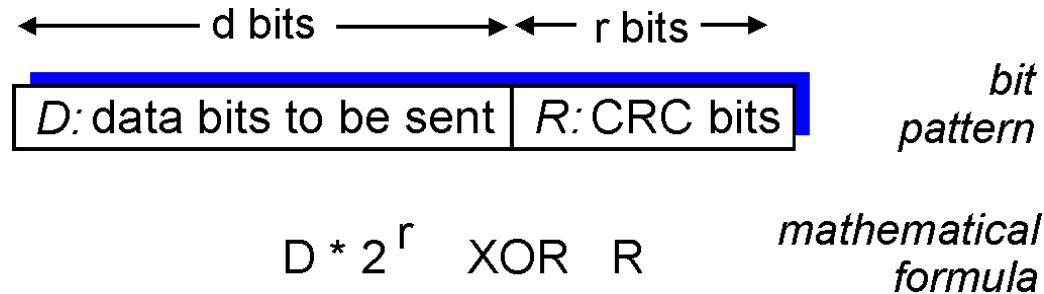
- Beispiele
 - Fehlererkennender Code
 - > Code mit einem einzigen Paritätsbit
 - > Hamming-Abstand des Codes = 2
 - > Änderung eines einzelnen Bits erfordert Änderung des Paritätsbits)
 - > Erkennung eines 1-Bit-Fehler möglich
(tatsächlich sogar Fehler mit ungerader Anzahl von Bits)
 - Fehlerbehebender Code (vereinfacht)
 - > 00000 00000, 00000 11111,
11111 00000, 11111 11111
 - > Hamming-Abstand des Codes = 5
 - > Korrektur von 2-Bit-Fehlern möglich
 - > Beispiel: 00000 00111 $\Rightarrow$ 00000 11111
 - > **FEC** (Forward Error Correction)

Eine schnelle Hardware-Lösung Cyclic Redundancy Check

- Polynomcodes: Interpretiere Datenbits **D** als Bitkette eines Polynoms, dessen Koeffizienten die 0-1-Werte der Bitkette sind

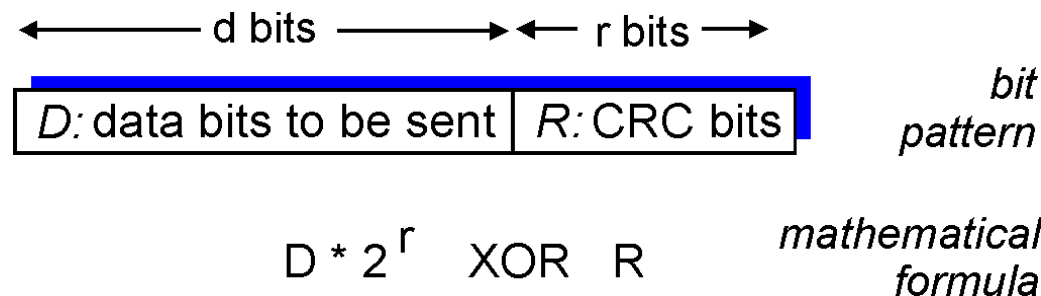
$$1\ 0\ 1\ 1 \rightarrow u^3 + u + 1$$

Prüfung der Nachricht mittels CRC basiert auf Polynomarithmetik



Prüfsummen: Cyclic Redundancy Check

- Sender und Empfänger einigen sich auf ein gemeinsam verwendetes Bitmuster der Länge $r+1$ Bit, das als **Generator G** bezeichnet wird. **Das höchstwertige Bit hiervon ist 1**
- Konzept: Berechne die r CRC-bits **R** so, dass die $d + r$ Bits (als Binärzahl interpretiert) mit der **Modulo-2-Arithmetik** genau durch G teilbar sind
 - Empfänger kennt G , teilt $\langle D, R \rangle$ durch G . Falls der Rest ungleich 0, so liegt ein Fehler vor!
 - Ethernet CRC kann Burst-Fehler von weniger als $r+1$ Bits und jede ungerade Fehlerzahl erkennen, aber keine Fehler beheben!



Modulo-2 Arithmetik

Die Rechenoperationen werden in der Modulo-2-Arithmetik einfacher, da hierbei keine Überträge zu berücksichtigen sind!

Addition und Subtraktion führen so zu dem gleichen Ergebnis.

Wir können einfach mit XOR arbeiten



Wie kann R bei bekanntem G berechnet werden?

Es gibt eine Bitkombination, so dass gilt:

$$D \cdot 2^r \text{ XOR } R = nG$$

D.h., R soll so gewählt werden, dass G in $D \cdot 2^r$ ohne Rest teilbar ist



Damit kann man R berechnen, denn wenn man $D \cdot 2^r$ durch G teilt, ist der Rest des Wertes genau R

$$R = \text{Rest}\left[\frac{D \cdot 2^r}{G}\right]$$

Generator G: 1001

Zu übertragende Daten:

101110

101110000

1001

001010000

1001

0011000

1001

01010

1001

0011

R Nullen!

R ist somit 011 !!!

Aufgabe

- Wir betrachten das CRC-Verfahren am Beispiel des Generatorpolynoms $x^4 + x^3 + 1$. Berechnen Sie die CRC-Prüfsumme zur Bitfolge 10110101110



Lösung

Wir berechnen $101101011100000:11001$

101101011100000

11001

011111011100000

11001

00110011100000

11001

0000011000

11001

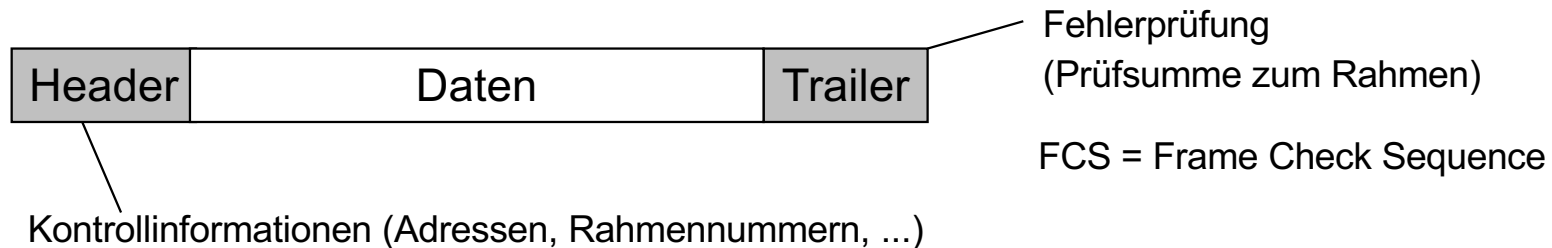
0000100

R ist somit 0100 !!!



Rahmenerstellung in der Sicherungsschicht

- Einteilung einer Nachricht in einheitliche Einheiten (einfachere Übertragung)
- wohldefinierte Schnittstelle nach oben (Schicht 3)
- Kennzeichnung der Einheiten:



Fehlererkennung: **ARQ** (Automatic Repeat Request)

- Verwendung eines fehlererkennenden Codes (CRC)
- Fehler werden erkannt, können aber nicht korrigiert werden! Daher müssen verfälschte Daten neu angefordert werden.
- Einführung einer **Flusskontrolle** (wie bei TCP: Sliding Window):
 - Identifikation der Datenblöcke
 - Quittierung von Blöcken durch den Empfänger
 - Wiederholung fehlerhaft übertragener Blöcke

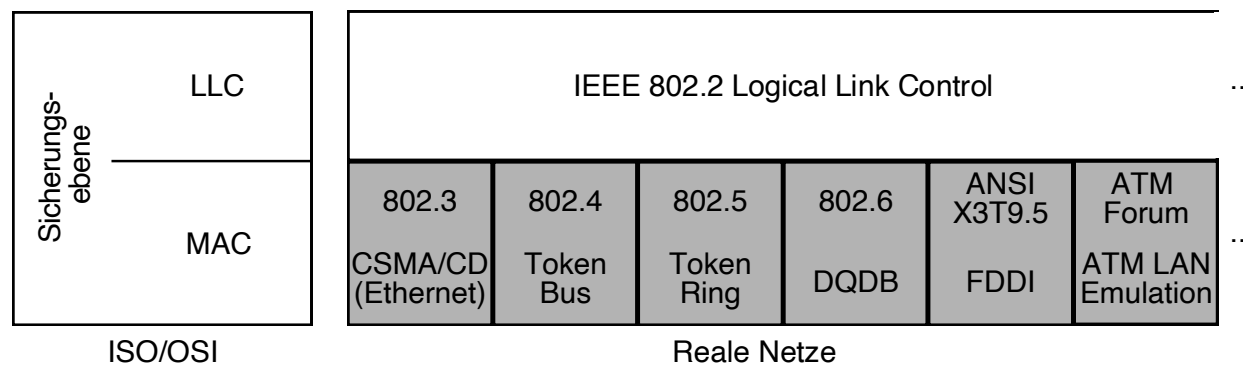
Schicht 2: Aufteilung in zwei Aufgabenbereiche

- **Logical Link Control (LLC)** (Schicht 2b)

- Einteilung der zu sendenden Daten in *Rahmen (Frames)*
- Bereitstellung einer (möglichst) fehlerfreien Übertragung zwischen benachbarten Knoten durch:
 - *Erkennung (und Behebung) von Übertragungsfehlern*
 - *Flusskontrolle* (Vermeidung der Überlastung des Empfängers)
 - *Pufferspeicher*

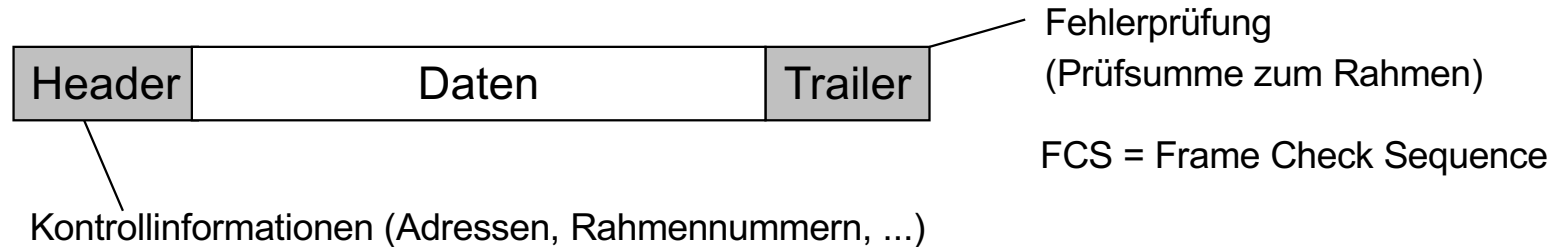
- **Medium Access Control (MAC)** (Schicht 2a)

- Regelung des Zugriffs auf den Kommunikationskanal bei Broadcast-Netzen



LLC: Rahmenerstellung

- Einteilung einer Nachricht in einheitliche Einheiten (einfachere Übertragung)
- wohldefinierte Schnittstelle nach oben (Schicht 3)
- Kennzeichnung der Einheiten:



Nächste Aufgabe der LLC-Schicht: sichere Übertragung der Rahmen zum Kommunikationspartner.

Die Übertragung über Schicht 1 ist nicht unbedingt fehlerfrei! Durch die Abschwächung der Signale bei der Übertragung oder durch Störungen „erkennt“ der Empfänger möglicherweise einige Bits falsch!